C17 Memory Safety, Pointer Arithmetic & Sanitizers • STUDENT EXAM COPY

Course: C 17 • College: VIT • Student Name: _____

Q1. Why is deterministic output normalization critical in automated code verification for C?

A) It masks memory addresses, timestamps, and thread IDs so identical logic yields identical results

B) It rewrites slow algorithms into faster ones automatically

C) It disables unit testing to make test suites run faster

D) It converts console output into binary machine code

Q2. What is the primary objective of mutation testing in exercise validation?

A) Proving that student-facing test suites successfully fail when code logic is intentionally mutated/broken

B) Randomly deleting student files to test backup recovery

C) Speeding up Docker container build times

D) Encrypting source code with AES-256

Q3. In C, what is the main consequence of executing untrusted code without container sandbox isolation?

A) Direct host filesystem compromise, resource exhaustion, and external network leaks

B) Syntax highlighting colors changing randomly

C) The compiler running 100x faster than expected

D) PowerPoint slide fonts changing to Comic Sans

Q4. When configuring container limits for isolated verification, why is '--network none' strictly enforced?

A) To prevent code from downloading malicious payloads, leaking credentials, or contacting external servers

B) Because Docker containers cannot support networking on Linux

C) To decrease the size of Docker image layers on disk

D) To disable JSON serialization in the API

Q5. What is the recommended approach for handling deprecation warnings in current production C?

A) Treat warnings as errors in CI/CD pipelines and migrate to the approved modern replacements

B) Pin older runtime versions indefinitely to avoid updating

C) Turn off compiler error logging across the entire company

D) Use reflection to bypass deprecated method signatures

Q6. Which practice is recommended when engineering scalable C applications for C17 Memory Safety, Pointer Arithmetic & Sanitizers?

A) Use verified idioms, explicit types, and automated unit test assertions

B) Suppress all compiler and deprecation warnings to simplify logs

C) Rely entirely on global mutable state to avoid passing parameters

D) Execute all database calls synchronously inside UI or main thread loops

Q7. Which practice is recommended when engineering scalable C applications for C17 Memory Safety, Pointer Arithmetic & Sanitizers?

A) Use verified idioms, explicit types, and automated unit test assertions

B) Suppress all compiler and deprecation warnings to simplify logs

C) Rely entirely on global mutable state to avoid passing parameters

D) Execute all database calls synchronously inside UI or main thread loops

Q8. Which practice is recommended when engineering scalable C applications for C17 Memory Safety, Pointer Arithmetic & Sanitizers?

A) Use verified idioms, explicit types, and automated unit test assertions

B) Suppress all compiler and deprecation warnings to simplify logs

C) Rely entirely on global mutable state to avoid passing parameters

D) Execute all database calls synchronously inside UI or main thread loops

Q9. Which practice is recommended when engineering scalable C applications for C17 Memory Safety, Pointer Arithmetic & Sanitizers?

A) Use verified idioms, explicit types, and automated unit test assertions

B) Suppress all compiler and deprecation warnings to simplify logs

C) Rely entirely on global mutable state to avoid passing parameters

D) Execute all database calls synchronously inside UI or main thread loops

Q10. Which practice is recommended when engineering scalable C applications for C17 Memory Safety, Pointer Arithmetic & Sanitizers?

A) Use verified idioms, explicit types, and automated unit test assertions

B) Suppress all compiler and deprecation warnings to simplify logs

C) Rely entirely on global mutable state to avoid passing parameters

D) Execute all database calls synchronously inside UI or main thread loops

Q11. Which practice is recommended when engineering scalable C applications for C17 Memory Safety, Pointer Arithmetic & Sanitizers?

A) Use verified idioms, explicit types, and automated unit test assertions

B) Suppress all compiler and deprecation warnings to simplify logs

C) Rely entirely on global mutable state to avoid passing parameters

D) Execute all database calls synchronously inside UI or main thread loops

Q12. Which practice is recommended when engineering scalable C applications for C17 Memory Safety, Pointer Arithmetic & Sanitizers?

A) Use verified idioms, explicit types, and automated unit test assertions

B) Suppress all compiler and deprecation warnings to simplify logs

C) Rely entirely on global mutable state to avoid passing parameters

D) Execute all database calls synchronously inside UI or main thread loops

Q13. What is the exact output produced by executing this C code?

```
print(10 * 4 + 2)
```

A) Compilation Error / Syntax Warning

B) 42

C) null

D) Runtime Exception: IndexOutOfBounds

Q14. What is the exact output produced by executing this C code?

```
print('Verified'.lower() + '-2026')
```

A) Compilation Error / Syntax Warning

B) verified-2026

C) null

D) Runtime Exception: IndexOutOfBounds